INTRODUCTION

Greenway Health APIs - An Overview

Options for Patients

FHIR

GAPI

TOOLS

Smart-on-FHIR Developer Dashboard

SMART App Launcher

FHIR Client Libraries

GAPI Developer Dashboard

Installing curl

CONTACT US

FHIR Support

MORE RESOURCES

Terms of Service

Greenway Health APIs - An Overview

Suggest Edits

TABLE OF CONTENTS

Introduction

What is an API and why would I use one?

Fast Healthcare Interoperability Resources (FHIR)

Greenway API (GAPI)

The Developer Platform

Introduction

The Greenway Health Developer Platform enables you to leverage Application Programming Interfaces (APIs) to build and deploy applications that work with clinical data stored in Greenway Health electronic health record (EHR) systems, such as Intergy and Prime Suite. These applications can be embedded inside the EHR, taking advantage of user and patient context, to provide clinicians with functionality at the point of care, or they can function on their own to provide functionality directly to users, independent of the EHR.

Greenway Health offers developers two APIs to connect to and interact with our products; a standards-compliant FHIR R4 API, as well as our proprietary GAPI. Documentation and tools supporting both are accessible right here on the Developer Platform.

What is an API and why would I use one?

An Application Programming Interface (API) is a way for two or more computer programs to communicate with each other. It is a type of software interface, offering a service to other pieces of software.

From a healthcare perspective, patients or healthcare providers (such as doctors, nurses. etc.) can use apps that communicate via an API to connect to the provider's medical practice data.

If you are a patient and a developer, you might have an idea for an app that you want to build for your own use to consolidate and or analyze your own medical data. Or perhaps there is an app out there that you want to use or adapt. As a patient of a provider that uses a Greenway EHR such as Intergy or Prime Suite, you can use your Greenway Patient Portal (GPP) or SmartHub login credentials to register here on the Developer Platform to learn the FHIR API and build your own application. When you run your application, you use your GPP/Smarthub login to enable your app to access your own data stored in your medical record via the FHIR API.

If you are a clinician and/or developer that uses of one of our EHR products in your practice, an API enables you to build applications that enable capabilities like syndromic surveillance, clinical registry reporting, population health, connect internal applications and much more. To build and add your own apps using the FHIR API, you use your Greenway Identity Services login to register here on the Developer Platform. The apps you publish will be visible to users inside your practices. Your users then use their Greenway Identify Services login to use your application to access your practice data via the FHIR API.

If you are a Greenway partner and/or a 3rd party developer writing new applications or adapting them to our platform, you can obtain your own login by registering on the Greenway Health Developer Platform and you can submit your app for addition to our App Gallery. Once your app is published, Greenway EHR users and Patients use their Greenway Identity Services logins to access their clinical data.

As previously stated, Greenway Healthcare has two APIs, described below.

Fast Healthcare Interoperability Resources (FHIR)

FHIR is a *Standards-based* API. The API uses FHIR Release 4 and is consistent for all of our EHR products. Our FHIR endpoints serve data from Greenway's core products, Intergy and Prime Suite.

In FHIR, all data entities are called *Resources*.

FHIR resources are structured in a familiar way - using nomenclature that is familiar to both clinicians and developers; nearly all of which is linked to a patient, which is expressed as a *Patient* resource.

The image below (published by the HL7 FHIR project), illustrates the different clinical resources that a FHIR server can support such as Allergy, Care Plan/Goal, Family History, and so on. Greenway's FHIR API provides a subset of these resources that is compliant with the 21st Century Cures Act requirements. Over time, support for more FHIR resources will be added to the platform.

Resources provided by FHIR R4

FHIR uses a REST (Representational State Transfer) API model that maps the different API transactions, such as creates, reads, updates, and deletes (developers affectionately refer to these actions as 'CRUD' operations), directly to HTTP methods like POST, GET, PUT, and DELETE. (Note: The current Greenway FHIR API supports read operations only at the present time.)

To facilitate the authentication apps and users to the FHIR API, the SMART-on-FHIR standard is used. SMART-on-FHIR uses the OAuth2 standard.

Greenway API (GAPI)

The Greenway API is a *Proprietary* API with separate and distinct API calls and data structures for each of our EHR products.

The GAPI supports reads and writes across a variety of clinical and financial data elements.

The Developer Platform

This platform is a central location where developers can learn about our API offerings and create applications.

Navigating within the Greenway Health Developer Platform is easy. In the top navigation bar, there are four links that take you to key areas.

FHIR Dashboard – Once you register as a developer, this area enables you to immediately begin building and deploying SMART-on-FHIR applications.
GAPI Dashboard – Our existing GAPI dashboard accessible to Marketplace partners and clients.
API Documentation – Technical information that describes our APIs. (No registration login is required to view our documentation.)
App Gallery – An application catalog for SMART-on-FHIR applications.

Updated almost 2 years ago

Options for Patients →

Did this page help you? Yes No

© 2023 All rights reserved. Greenway Health, LLC - Terms of Service - Privacy Policy

INTRODUCTION

Greenway Health APIs - An Overview

Options for Patients

FHIR

GAPI

TOOLS

Smart-on-FHIR Developer Dashboard

SMART App Launcher

FHIR Client Libraries

GAPI Developer Dashboard

Installing curl

CONTACT US

FHIR Support

MORE RESOURCES

Terms of Service

Options for Patients

Suggest Edits

Important

Patients who have appointments after May 10, 2024, should expect to view their data in FHIR. However, for those unable to do so, do the following:

- Reach out to your practice and have your provider create a *mock* visit to load your data
- If the practice is not able to add a *mock* visit (this can happen if you are deactivated, the practice refuses for an unspecified reason, etc.), you can request a copy of your record(s) at records.greenwayhealth.com, provided your provider uses the Intergy or Prime Suite EHR system.

Updated 5 months ago

TABLE OF CONTENTS

Important

← Greenway Health APIs - An Overview

FHIR →

Did this page help you? Yes No

© 2023 All rights reserved. Greenway Health, LLC - Terms of Service - Privacy Policy

INTRODUCTION

Greenway Health APIs - An Overview

Options for Patients

FHIR

How to Create and Publish SMART-on-FHIR Apps

Overview - Authentication and Authorization

Authorization Scopes

Authorization Code Authentication using Python

Bulk Export

How to Register a SMART Backend Service

How to host a Javascript SMART-on-FHIR app on GitHub

GAPI

TOOLS

Smart-on-FHIR Developer

FHIR

Suggest Edits

Greenway Health's standards-compliant API

TABLE OF CONTENTS

Here for the first time?

More background

Here for the first time?

Check out these quick start guides:

- Create your first SMART-on-FHIR app
- FHIR Authentication and Authorization

> **FHIR Base URL**
>
> Click here for a complete list of customer endpoints.

Familiarize yourself with this documentation and be sure to sign up in order to be able to deploy your own apps.

If you have a Greenway Identity Services login either as a patient or as an EHR user, you can immediately interact with the API with familiar tools, such as cURL:

cURL

```
curl "https://fhir-api.fhirprod.aws.greenwayhealth.com/fhir/R4/{{TENANT_ID}}"
```

Authentication and authorization to our FHIR endpoints uses OAuth. Patients and users of our EHR products log in to our production endpoints using their Greenway Identity credentials. A public sandbox will be available soon.

Refer to our Authentication guide for more information.

Once authenticated, calls to the FHIR API look like this:

cURL Python

```
curl --request GET \
     --url 'https://fhir-api.fhirprod.aws.greenwayhealth.com/fhir/R4/[TENANT_ID]/Patient/1234' \
     --header 'accept: */*' \
     --header 'authorization: Bearer ThisIsATokenABC123'
```

More background

FHIR, which stands for Fast Healthcare Interoperability Resources, is organized around 'resources' or endpoints that represent the many different data types found in healthcare. Within each resource there are multiple data points. For example, Patient would include details about name, gender, date of birth, marital status, address and so on. The resource names are fairly straightforward. Here is a list of most of the resources available in our API:

- AllergyIntolerance
- Binary
- CarePlan
- CareTeam
- Condition
- Coverage
- Device
- DiagnosticReport
- DocumentReference
- Encounter
- Goal
- Immunization
- Location
- Medication
- MedicationRequest
- Observation
- Organization
- Patient
- Practitioner
- PractitionerRole
- Procedure
- Provenance

For much more detail, see the API Reference

Updated about 1 month ago

← Options for Patients

How to Create and Publish SMART-on-FHIR Apps →

Did this page help you? Yes No

© 2023 All rights reserved. Greenway Health, LLC - Terms of Service - Privacy Policy

INTRODUCTION
Greenway Health APIs - An Overview
Options for Patients
- FHIR
How to Create and Publish SMART-on-FHIR Apps
Overview - Authentication and Authorization
Authorization Scopes
Authorization Code Authentication using Python
Bulk Export
How to Register a SMART Backend Service
How to host a Javascript SMART-on-FHIR app on GitHub
- GAPI
TOOLS
Smart-on-FHIR Developer

How to Create and Publish SMART-on-FHIR Apps

Suggest Edits

TABLE OF CONTENTS
Introduction
Developer Registration
App Registration

Introduction

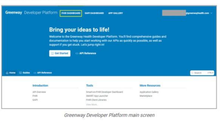
The Greenway Health Developer Platform makes building SMART-on-FHIR applications easy; enabling you to build and deploy applications that work with clinical data stored in Greenway Health electronic health record (EHR) systems such as Intergy and Prime Suite. Applications you build can be embedded inside the EHR, taking advantage of user and patient context, to provide clinicians with functionality at the point of care, or they can function on their own to provide functionality directly to users, independent of the EHR.

Developer Registration

Our FHIR API documentation is always available to the public - no registration or login is required.

Once you are ready to build and deploy SMART-on-FHIR apps, you can register on the Greenway Health Developer Platform by following these steps:

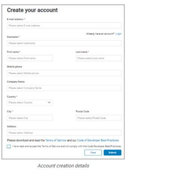
1. Go to the registration page.
   The **Create your account** screen appears.



Greenway Developer Platform main screen

2. Fill in your relevant details, read and accept the *Terms of Service*, select the *Acceptance* check box and select the **Submit** button.

Once you register, you are re-directed to the *Greenway Health Identity Services* authentication screen. In parallel, a PIN is sent to your email and will navigate you to the password form where you can set your password.



Account creation details

Greenway Health One Time Pin Email

Password confirmation

3. Once your *User Name* and *Password* are set, navigate back to the FHIR Dashboard, enter your credentials that you filled during registration and select the **Sign On** button.

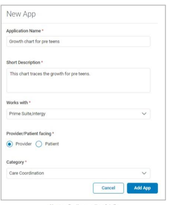
Greenway Health User Credentials

App Registration

1. Once you sign on using your credentials, the app screen appears. Select the **Add App** button.
   The **New App** screen appears, where you can fill in the mandatory fields.

Greenway Developer Platform Default Applications

> **Note**
> Initially, the Apps screen is empty as you are a first-time user and have not created any apps.

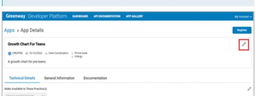


New application creation details

> **Note**
> - **Short description** should be between 15 to 250 characters.
> - For **Provider/Patient Facing** option, select:
> - **Provider**: if you are designing the app for a Doctor/Nurse/Person providing third party care services.
> - **Patient**: if you are designing the app for an individual.
>
> If you select **Provider**, you cannot edit the subsequent **App Details** screen. However, if you select **Patient**, you can edit the **App Details** screen and fill in details as required.

2. Once you fill in all relevant fields, select the **Add App** button.
   The **App Details** screen appears. Select the *pencil*icon to edit the **Technical Details** and **General information** sections, respectively.

> **Note**
> The **Technical Details** and **General Information** sections are mandatory while the **Documentation** section is optional.



Application details during registration

- Select the **Application Launch Type** based on the type of app that you are designing:
  - **Standalone** - The app is built for the Web/Desktop/Mobile.
  - **EHR embedded** - The app is built to run within the EHR, in one of two contexts:
    - *Runs inside a patient chart* - The app retrieves data only for a particular patient. In other words, the app is tied to a Patient Id.
    - *Runs outside a patient chart* - The app retrieves data for patients who are part of the EHR system, that is, the app retrieves data for multiple Patient IDs.
  - **Back-end Service** – The app does not have a user interface and it runs independently of any user interaction. (This is also used for Bulk FHIR.)
- Select the **Authentication Type**: if you are developing a javascript/HTML5 or mobile application, or are not sure, select Public. If the app logic will execute on a trusted server, select Confidential.
- Select the **Launch context**. If this is a standalone application that requires a patient, select launch/patient. If the standalone application also requires an encounter, select launch/encounter
- Select **Scopes**. If your application is intended for use with a single patient, select the **Scopes for Individual Patient** button followed by the resources you wish to enable. If the application is intended for use with multiple patients, select the **Scopes for Multiple Patients** button followed by the resources you wish to enable.

Patient scope details and Smart Launch URL

- When finished, go to the bottom of the screen and select **Save**.

3. Select **Register**
   The app is registered accordingly.

Select Register

Registered application

4. Select the pencil again and then enable the **Production** slider; typing in a URL that is unique to your application for both **Smart Launch URL** and **Redirect URL** sections, respectively. The **Smart Launch URL** will trigger the authentication process and navigate your app user to a *Login Screen* where your user provides their credentials. Once authenticated, the user redirects to the **Redirect URL**.

> **Note**
> If you want to test your app by directing the Smart Launch URL and Redirect URL to your own machine while you develop, you can add an entry to your hosts file, or use a DNS alias service for localhost such as *lvh.me*. You can, of course, host your application anywhere you choose. One helpful option for this while you develop your app is GitHub.

5. Select **Submit for Review**.
   A confirmation popup displays accordingly.

Confirmation popup

6. Select **Submit for Review** in the popup message.
   An **Application Submission Successful** message displays accordingly.

Application Submission Successful

> **Note**
> Once you submit your application for review, in the backend, the Greenway Health admin checks your submitted app for *Technical Details, General Information* and *Documentation* that supports your application. In case the admin is not satisfied with this information or feels that some correction is required for your submitted application, you will receive an email to this effect asking for the information to be provided or corrected.
>
> Once the admin is satisfied that you have provided all relevant information for the submitted application, it is published in the **App Gallery** page of Greenway Health.

Application published in the app gallery

You can select the **More Information** link below any of the apps in the list to view its details.

Link for More Information

Application details

> **Note**
> You can also try out the applications on display in the app gallery. To do so, select **Tryit**.

Try the application to test it

> **Note**
> You can also deactivate an app that you create. To do so, select the **Deactivate** button at the top right of the **App Details** screen.

Deactivate an application

Updated about 1 year ago

Did this page help you? Yes No

© 2023 All rights reserved. Greenway Health, LLC - Terms of Service - Privacy Policy

INTRODUCTION

Greenway Health APIs - An Overview

Options for Patients

FHIR

How to Create and Publish SMART-on-FHIR Apps

Overview - Authentication and Authorization

Standalone Authorization Walkthrough

Authorization Scopes

Authorization Code Authentication using Python

Bulk Export

How to Register a SMART Backend Service

How to host a Javascript SMART-on-FHIR app on GitHub

GAPI

Overview - Authentication and Authorization

Suggest Edits

TABLE OF CONTENTS

About the process

How it works

See it in action

About the process

At a high level, authentication of a SMART-on-FHIR application uses OAuth authentication. For users, this is usually the 'authorization code' workflow which broadly comprises gathering several pieces of information including login credentials and user permissions, and then submitting that information to a security endpoint in order to obtain an OAuth Access Token and a Refresh Token. These tokens enable the app to access clinical data.

How it works

Launch and Endpoint Discovery
In a typical scenario, a SMART-on-FHIR app that has been registered on the Developer Platform first obtains the URL for the security endpoint to which the app must connect to authenticate. This information is retrieved from the conformance statement that is served from the FHIR server's `/metadata` endpoint.

Authentication and Consent
Once the app has the security endpoint URL, the authentication process can begin. The app identifies itself to the security endpoint using the unique identifier that was assigned to it when it was registered on the Developer Platform (this is called a *Client ID*), and then the app requests permission to access the FHIR resources it requires – these permissions are referred to as scopes.

The security endpoint responds with a screen that enables the user of the app to enter their login credentials and grant consent to the permission scopes that the app requested.

Authorization
Once the user authenticates and grants consent to the requested scopes, the security endpoint issues two tokens:

An *Access Token*, which is submitted by the app to the FHIR endpoint in the header of every FHIR request the app makes; as proof that the app and user are permitted to access the FHIR endpoint and data. This token has a fixed expiration date, beyond which it will no longer enable access to the FHIR endpoint.

A *Refresh Token*, which is used by the app (without requiring any additional user interaction) to obtain a new Access Token from the security endpoint once the current Access Token expires. The length of the Refresh Token's validity varies based on whether the user consented to permit the app to remain authenticated during a single user session ('online_access' scope), or permit the authentication to last across multiple sessions ('offline_access' scope).

Whenever a new Access Token is issued, a new Refresh Token is also issued. Once the Refresh Token is no longer valid, the app must prompt the user to reauthenticate. This restarts the authentication process from the beginning.

See Also: SMART App Launch Framework

See it in action

We are currently in the process of creating:

- For a detailed walkthrough of standalone authorization along with the source code, refer to the Standalone Authentication Walkthrough page
- A place which contains a source code for the tutorial. This is particularly useful for those of you who would like to deploy the code yourself. Note that in such a case, you will need to Publish it via your dashboard and use your own client ID and tenant ID.

Updated about 1 month ago

← How to Create and Publish SMART-on-FHIR Apps

Standalone Authorization Walkthrough →

Did this page help you? Yes No

© 2023 All rights reserved. Greenway Health, LLC - Terms of Service - Privacy Policy

INTRODUCTION
Greenway Health APIs - An Overview
Options for Patients
FHIR
How to Create and Publish SMART-on-FHIR Apps
Overview - Authentication and Authorization
Standalone Authorization Walkthrough
Authorization Scopes
Authorization Code Authentication using Python
Bulk Export
How to Register a SMART Backend Service
How to host a Javascript SMART-on-FHIR app on GitHub
GAPI

Standalone Authorization Walkthrough

Suggest Edits

TABLE OF CONTENTS

Step 1: Launch
Step 2: Endpoint Discovery
Step 3: Get Authorization Code
Step 4: Acquire Access Token
Step 5: Retrieve Data

The app launch URLs mentioned below are sample URLs that have no functionality. Use your own URL for app launch and redirect URLs. For endpoints, refer to FHIR Base URLs.

Step 1: Launch

When you launch this app, a special URI is used which is shown here. The 'Endpoint' value refers to the FHIR endpoint from which data will be retrieved.

Title	Link
App launch URL	[https://mysmartonfhirapplication.mydomain.com/standalone-authorization-tutorial/launch.html]
End point	[https://fhir-api.fhirprod.aws.greenwayhealth.com/fhir/R4/{tenant-id}]

Step 2: Endpoint Discovery

In this step, your app obtains the Conformance Statement from the FHIR endpoint. This file contains the authentication servers from which your app obtains the authentication code which it then exchanges for the token.

Method	End Point
Get Discovery Endpoint (Conformance Statement):	[https://fhir-api.fhirprod.aws.greenwayhealth.com/fhir/R4/{tenant-id}/metadata]
Authorization Endpoint:	[https://auth-api.login.greenwayhealth.com/oauth2/auth-code]
Token Endpoint:	[https://auth-api.login.greenwayhealth.com/oauth2/as/token.oauth2]

Step 3: Get Authorization Code

In this first half of the authorization, your app identifies itself to the authorization code endpoint using its client ID and the scopes it desires in order to access clinical data. Your app also provides a state value that identifies the session context that your app establishes with the authorization server. This is a security measure to protect against Cross Site Request Forgery (CSRF).
In response, the authorization server displays a consent screen. Once the user grants consent, the authorization server provides your app with an authorization code.

Method	End Point
Get Authorization Endpoint	[https://auth-api.login.greenwayhealth.com/oauth2/auth-code]

URL Parameters

URL Parameters	
client_id	MGMzYzliNjYtNzI2Yi00MmI3LThmNzUtYmMyMTkzN2I0ZWQ2
response_type	code
scope	launch/patient openid fhirUser online_access patient/Patient.read patient/Observation.read
state	kawltDHKN7
Returned Authorization Code	QqmMdC19nQ4TdckElFvOSWnp0NVUNTqIWno78iFe

Step 4: Acquire Access Token

This is the second half of the authorization process. After authenticating the user and getting user consent for the scopes, the authorization server has redirected the user back to your app; passing it an authorization code and the state parameter it received from your app.

```
HTTP

<https://mysmartonfhirapplication.mydomain.com/standalone-authorization-tutorial/launch.html>/sta
```

First, your app should verify that the state value received matches what you sent in the previous step to protect against CSRF attacks.

If it matches, your app posts the code obtained in Step 3 along with the parameters below, and in exchange, your app receives a token that it uses to authenticate itself to receive FHIR data, as well as a number of claims that enable your app to function, such as patient.

```
JSON

POST Token Endpoint
```

POST Request

```
JSON

POST https://token-endpoint
...
```

POST Parameters	
authorization_code	QqmMdC19nQ4TdckElFvOSWnp0NVUNTqIWno78iFe
grant_type	authorization_code
redirectUri	[https://mysmartonfhirapplication.mydomain.com/standalone-authorization-tutorial/launch.html/callback.html]
client_id	MGMzYzliNjYtNzI2Yi00MmI3LThmNzUtYmMyMTkzN2I0ZWQ2

POST Response

```
Text

{

}
```

Response Parameters	
access_token	**Show**
token_type	Bearer
expires_in	7199
scope	patient/Observation.read
patient	1

Step 5: Retrieve Data

Your app uses the token and the *patient* claim received in the previous step to call the FHIR server to retrieve data.

GetFHIR API:	
Auth Header	Bearer {token}
Authorization	**Show**

Updated 27 days ago

Overview - Authentication and Authorization

Authorization Scopes

Did this page help you? Yes No

© 2023 All rights reserved. Greenway Health, LLC - Terms of Service - Privacy Policy

INTRODUCTION

Greenway Health APIs - An Overview

Options for Patients

FHIR

How to Create and Publish SMART-on-FHIR Apps

Overview - Authentication and Authorization

Authorization Scopes

Authorization Code Authentication using Python

Bulk Export

How to Register a SMART Backend Service

How to host a Javascript SMART-on-FHIR app on GitHub

GAPI

TOOLS

Smart-on-FHIR Developer

Authorization Scopes

Suggest Edits

TABLE OF CONTENTS

What are scopes?

Greenway FHIR API Scopes

What are scopes?

In SMART-on-FHIR (and OAuth in general) a 'scope' is a way to limit an app's access to a user's data. Rather than granting complete access to a user's clinical data, it is preferable to give apps a way to request a more limited scope of what data they are they are allowed to access.

Some apps only need to identify the user and retrieve patient demographics, so they only need access to a user ID and basic identifying information. Other apps may need to know more information such as a patient's vital signs, or they may need broad access to an entire population and the ability to retrieve all medications and lab results.

The idea is that users will be more willing to authorize an application if they know exactly what the application can and cannot do with the data. Scopes are a way to control access and help the user identify the permissions they are granting to the application.

It's important to remember that scopes are not the same as the internal permissions system of Greenway Intergy, Prime Suite, Patient Portal and SmartHub. Scopes are a way to limit what a SMART-on-FHIR application can do within the context of what a user can do.

For example, if you have a user that is not permitted to view clinical information in the EHR, and the application is requesting the `user/Patient.read` scope, the Authorization server is not going to permit calls to the `/Patient` resource to retrieve all Patients in the practice because that user is not allowed to access that data themselves.

Another way to think of this is that 'scopes' are the permissions that the SMART-on-FHIR application requests from the user that wants to use the app.

We support the following scopes:

Greenway FHIR API Scopes

Scopes

```
openid - Permission to retrieve information about the current logged-in user.
fhirUser - Permission to retrieve information about the current logged-in user.
online_access - Request a refresh_token that can be used to obtain a new access token to replac
offline_access - Request a refresh_token that can be used to obtain a new access token to repla
launch - Permission to obtain launch context when app is launched inside the EHR.
launch/patient - When launching outside the EHR, ask for a patient to be selected at launch tim
launch/encounter -  When launching outside the EHR, ask for an encounter to be selected at laun
patient/AllergyIntolerance.read - Permission to read this resource for the current patient
patient/Appointment.read - Permission to read this resource for the current patient
patient/Binary.read - Permission to read this resource for the current patient
patient/CarePlan.read - Permission to read this resource for the current patient
patient/CareTeam.read - Permission to read this resource for the current patient
patient/Condition.read - Permission to read this resource for the current patient
patient/Consent.read - Permission to read this resource for the current patient
patient/Coverage.read - Permission to read this resource for the current patient
patient/Device.read - Permission to read this resource for the current patient
patient/DiagnosticReport.read - Permission to read this resource for the current patient
patient/DocumentReference.read - Permission to read this resource for the current patient
patient/Encounter.read - Permission to read this resource for the current patient
patient/Goal.read - Permission to read this resource for the current patient
patient/Group.read - Permission to read this resource for the current patient
patient/Immunization.read - Permission to read this resource for the current patient
patient/Location.read - Permission to read this resource for the current patient
patient/Medication.read - Permission to read this resource for the current patient
patient/MedicationRequest.read - Permission to read this resource for the current patient
patient/Observation.read - Permission to read this resource for the current patient
patient/Organization.read - Permission to read this resource for the current patient
patient/Patient.read - Permission to read this resource for the current patient
```

See also SMART App Launch: Scopes and Launch Context

Updated about 2 years ago

← Standalone Authorization Walkthrough

Authorization Code Authentication using Python →

Did this page help you? Yes No

© 2023 All rights reserved. Greenway Health, LLC - Terms of Service - Privacy Policy

INTRODUCTION

Greenway Health APIs - An Overview

Options for Patients

FHIR

How to Create and Publish SMART-on-FHIR Apps

Overview - Authentication and Authorization

Authorization Scopes

Authorization Code Authentication using Python

Bulk Export

How to Register a SMART Backend Service

How to host a Javascript SMART-on-FHIR app on GitHub

GAPI

TOOLS

Smart-on-FHIR Developer

Authorization Code Authentication using Python

Suggest Edits

TABLE OF CONTENTS

Set up and publish a SMART on FHIR client

Get an authorization code

Trade the authorization code for a token

Use the token

To work with the FHIR API you first need to authenticate the application and the user via SMART-on-FHIR OAuth. The process happens in a few simple steps:

- **Set up and publish a SMART-on-FHIR client**.
- **Get an authorization code**.
- **Trade the authorization code for a token**.
- **Use the token to access the FHIR API**.

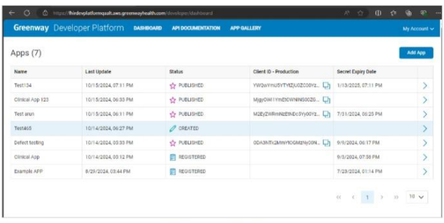
Once you complete these steps, you'll be able to execute the actions made by users of your SMART-on-FHIR application. We've provided more detail and some sample Python code to show you how this works.

Set up and publish a SMART on FHIR client

A detailed step-by-step guide is here in the App Registration section.

Get an authorization code

Once the app is published, copy the client ID for the app which can be found on the Apps list on the Developer Dashboard.



Dev Platform App Dashboard

When the app is launched by the user, to get an authorization code, the user clicks a link that triggers a GET request to the following address, with your app's client ID and other info filled in instead of the blanks. (We've got some sample code you can use to do this further down in this guide.)

```
curl
https://auth-api.login.greenwayhealth.com/oauth2/auth-code?response_type=code&client_id=___&scope=
```

The address is our Authorization server. You can always obtain this address by querying the FHIR server's /.well-known/smart-configuration or /metadata endpoint in any API tool or by using the following cURL command.

Greenway Health's FHIR servers return the /.well-known endpoints as a format most web browsers will not recognize. Instead, please query this end point via API call to obtain the details.

```
curl
https://fhir-api.fhirprod.aws.greenwayhealth.com/fhir/R4/{tenant_id}/. well-known/smart-configura
```

> You cannot query directly using a web browser.

The abovementioned `auth-code` request will trigger a user login and, once the user is authenticated, will result in a consent screen that presents the user with the scopes (permissions) that your app requires. (You declared these scopes on the Developer Platform App Dashboard when you created the app). Once the user approves the scopes request, they are redirected back to the Redirect URL you provided on the App Dashboard when you created the app, and the app then retrieves a token (we will see that in the next step).

Here is some sample code that you can use to authorize a request [LINK]. It's a Python/Flask app that generates a token which is available in the console and browser. You can then add your own code to the sample to use the token, or simply access the functionality as is, using the browser and copying the token for use with the command line tools like cURL.
To run the sample, you will need to generate a Self-signed Certificate and a Private Key (if you don't already have SSL set up)

```
Shell
openssl req -x509 -nodes -days 365 -newkey rsa:1024 -keyout pkey.pem -out cert.crt
```

Once you have the SSL set up you can configure and then run the sample Flask app with our security endpoints to test the OAuth flow.

Be sure to update the information in the Flask app.

Here is an example of what the `auth-code` link looks like. (This is the same link as found in the "Authenticate" button in the `launch.html` file that the Flask app serves):

```
HTML
<a href="https://auth-api.login.greenwayhealth.com/oauth2/auth-code?client_id=XXXXXXXXXXXX&respons
```

Once you run the Flask app, click the button through to the authentication screen and then authenticate, you'll see the auth code in the first few lines output on the terminal similar to the output below:

```
Shell
$ python3 ssl_server.py
 * Serving Flask app 'ssl_server'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production
 * Running on all addresses (0.0.0.0)
 * Running on https://127.0.0.1:4567
 * Running on https://192.168.255.206:4567
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 823-965-749
192.168.255.206 - - [06/Sep/2022 17:28:41] "GET / HTTP/1.1" 200 -
192.168.255.206 - - [06/Sep/2022 17:28:41] "GET /AuthButton.png HTTP/1.1" 200 -
CODE: AXVYHSAhFvofZW-A5LOuyJ-b8JV3u1SfhBA59mFp
STATE: 42f8cdab
```

Trade the authorization code for a token

The app then exchanges the authorization code for an OAuth access token.
The full output on the terminal:

```
Shell
$ python3 ssl_server.py
 * Serving Flask app 'ssl_server'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production
 * Running on all addresses (0.0.0.0)
 * Running on https://127.0.0.1:4567
 * Running on https://192.168.255.206:4567
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 823-965-749
192.168.255.206 - - [06/Sep/2022 17:28:41] "GET / HTTP/1.1" 200 -
192.168.255.206 - - [06/Sep/2022 17:28:41] "GET /AuthButton.png HTTP/1.1" 200 -
CODE: AXVYHSAhFvofZW-A5LOuyJ-b8JV3u1SfhBA59mFp
STATE: 42f8cdab
ACCESS TOKEN: eyJhbGciOiJSUzI1NiIsImtpZCI6IjNxMC1RMElkd1ltRnhzNlNZVXRPbEdrTnRhTSIsInBpLmF0bSI6InZ
```

Behind the scenes above, the app has performed a POST to the token endpoint with the authorization code it received from the auth-code request

Use the token

Once you have the authorized token you can use it in http calls made in your own Python code to the FHIR API to retrieve data.

> **A note about Public apps, PKCE, and Confidential apps**
>
> Using OAuth and SMART-on-FHIR, we authenticate both the client (via the client ID) and the user (via their login). The example we showed here is a generic authorization code workflow for what is considered a *Public* client application.
>
> *Public* applications normally run on the user's device and do not have a back-end server. Storing a shared secret on the user's device to authenticate a *Public* client would not be appropriate; as a user could, in theory, retrieve the secret. As an extra layer of security, we optionally support the Proof Key for Code Exchange (PKCE) which provides a measure of security that compensates for the lack of a client secret and protects against request forgery attacks that intercept the authorization code. Use of PKCE would entail use of additional parameters, `code_challenge` and `code_challenge_method=S256` on the `auth-code` call.
>
> As it happens, our Python application runs on a server, that - under normal circumstances - would most likely be properly secured, so we would likely designate it as a *Confidential* client. That is to say, the app can keep a client secret because the authentication exchange occurs on the back end between the Python server and the FHIR API, out of the reach of the user accessing the application in the browser. Instead of an authorization code, a *Confidential* application would possess a public/private key pair, whose public key is declared on the Developer Platform. Using its private key, the *Confidential* app would sign and submit a JWT token to the authentication server. We plan to release a separate tutorial for that, so stay tuned....

Updated 7 days ago

← Authorization Scopes

Bulk Export →

Did this page help you? Yes No

© 2023 All rights reserved. Greenway Health, LLC - Terms of Service - Privacy Policy

INTRODUCTION

Greenway Health APIs - An Overview

Options for Patients

FHIR

How to Create and Publish SMART-on-FHIR Apps

Overview - Authentication and Authorization

Authorization Scopes

Authorization Code Authentication using Python

Bulk Export

How to Register a SMART Backend Service

How to host a Javascript SMART-on-FHIR app on GitHub

GAPI

TOOLS

Smart-on-FHIR Developer

Bulk Export

Suggest Edits

TABLE OF CONTENTS

Step 1 - Export a Group of Patients
Request
Responses
Step 2 - Status Request
Request
Responses
Cancelling an Export Request
Request
Responses
Step 3 - File Retrieval
Request
Responses
See it in action

The FHIR Bulk Export mechanism is intended to enable properly authorized and authenticated clients to work with larger FHIR datasets representing multiple patients.

FHIR Bulk Export is authenticated using Backend Services Authorization and happens in three steps:

1. Client requests a bulk-data export,
2. Client receives status information regarding progress in the generation of the requested files,
3. Client retrieves the requested files.

Step 1 - Export a Group of Patients

The first operation initiates the asynchronous generation of data to which the client is authorized for a group of patients.
(See also API Reference - Group/(id)/$export)

Request

`GET [fhir base URL]/Group/[id]/$export`

Optional Parameters

Name	Type	Description
_since	date	Resources updated after this period will be included in the response. *If this parameter is absent only resources created or updated in the past 24 hours will be exported.*
_type	string	A string of comma-delimited FHIR resource type to be included in the response. If this parameter is omitted, the server returns all supported resources within the scope of the client authorization.

Responses

Success

- HTTP Status Code of `202 Accepted`
- `Content-Location` header with the absolute URL of an endpoint for subsequent status requests (polling location)
- Optionally, a FHIR OperationOutcome resource in the body

Error (e.g., unsupported search parameter)

- HTTP Status Code of `4XX` or `5XX`
- The body is a FHIR OperationOutcome resource in JSON format

Step 2 - Status Request

After a bulk data request has been started, the client can poll the status URL provided in the `Content-Location` response header of the $export request (Step 1).

Request

`GET [polling content location]`

Responses

In Progress

- HTTP Status Code of `202 Accepted`
- `X-Progress` header with a text description of the status of the request

Error

- HTTP Status Code of `4XX` or `5XX`
- `Content-Type` header of `application/json`
- The body is a FHIR OperationOutcome resource in JSON format

Complete

- HTTP Status Code of `200 Ok`
- `Content-Type` header of `application/json`
- A body containing a JSON object providing metadata and links to the generated bulk data files.

Example response body

```
{
    "transactionTime": "[instant]",
    "request" : "[base]/Patient/$export?_type=Patient,Observation",
    "requiresAccessToken" : true,
    "output" : [{
      "type" : "Patient",
      "url" : "http://serverpath2/patient_file_1.ndjson"
    },{
      "type" : "Patient",
      "url" : "http://serverpath2/patient_file_2.ndjson"
    },{
      "type" : "Observation",
      "url" : "http://serverpath2/observation_file_1.ndjson"
    }],
    "error" : [{
      "type" : "OperationOutcome",
      "url" : "http://serverpath2/err_file_1.ndjson"
    }],
    "extension":{"http://myserver.example.org/extra-property": true}
  }
```

Cancelling an Export Request

If needed, after a bulk data request has been started, a client can send a delete request to the URL provided in the `Content-Location` header (Step 1). to cancel the request.

Request

`DELETE [polling content location]`

Responses

Success

- HTTP Status Code of `202 Accepted`
- Optionally a FHIR OperationOutcome resource in the body

Error

- HTTP status code of `4XX` or `5XX`
- The body is a FHIR OperationOutcome resource in JSON format

Step 3 - File Retrieval

Using the URLs supplied by the FHIR server in the Complete Status response body (Step 2), the client can download the generated bulk data files (one or more per resource type). The file is returned in newline-delimited JSON format (NDJSON).

Example NDJSON output file


```
{"id":"5c41cecf-cf81-434f-9da7-e24e5a99dbc2","name":[{"given":["Brenda"],"family":["Jackson"]}],"
{"id":"3fabcb98-0995-447d-a03f-314d202b32f4","name":[{"given":["Bram"],"family":["Sandeep"]}],"ge
{"id":"945e5c7f-504b-43bd-9562-a2ef82c244b2","name":[{"given":["Sandy"],"family":["Hamlin"]}],"ge
```


Request

`GET [url from status request output field]`

Responses

Success

- HTTP status of `200 OK`
- `Content-Type` header `application/fhir+ndjson`
- Body of FHIR resources in newline delimited json

Error

- HTTP Status Code of `4XX` or `5XX`

See it in action

A publicly available Bulk Data client is available here.

Updated about 2 years ago

← Authorization Code Authentication using Python

How to Register a SMART Backend Service →

Did this page help you? Yes No

© 2023 All rights reserved. Greenway Health, LLC - Terms of Service - Privacy Policy

INTRODUCTION

Greenway Health APIs - An Overview

Options for Patients

FHIR

How to Create and Publish SMART-on-FHIR Apps

Overview - Authentication and Authorization

Authorization Scopes

Authorization Code Authentication using Python

Bulk Export

How to Register a SMART Backend Service

How to host a Javascript SMART-on-FHIR app on GitHub

GAPI

TOOLS

Smart-on-FHIR Developer

How to Register a SMART Backend Service

Suggest Edits

TABLE OF CONTENTS

Introduction

Enabling a Back-end Service

Introduction

SMART-on-FHIR Back-end Services can be used to autonomously access all FHIR resources via the FHIR API and FHIR Bulk Data API to enable key clinical functions such as population health analytics, electronic case reporting, connect to your internal applications, bulk data export, and more.

Once you enable and authorize an app to function as a Back End Service, the app usually functions autonomously, without further user authentication or interaction.

FHIR Back-end Applications and Intergy Multi-Practice Systems

If you have configured your Intergy instance to support multiple practices, you can designate which practice(s) the Back-end Service can access. The Back-end Service will then have access to the FHIR API data for the practices you've designated*.

There is no limit to the number of Back-end Services that you can create, so there is flexibility to support multiple practice configurations.

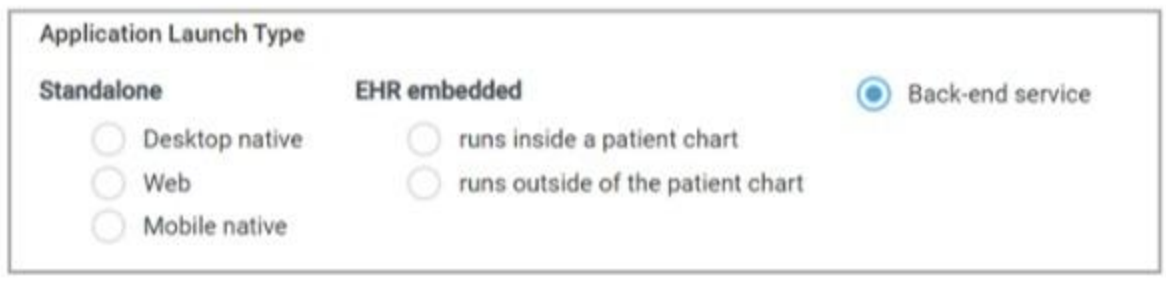
For example, if one of your practices uses a particular population health platform and another practice submits to a clinical registry for their specialty, two Back-end Services can be created to support these different workflows.

> ***Bulk Export Operations**
>
> Please note that bulk export operations via the Bulk Data API endpoint occur at the tenant (site) level by default, exporting data from all practices for the tenant. To export a subset of the data, filter parameters can be applied to the Bulk API request. Please refer to the Bulk Export API guide for more information.

Enabling a Back-end Service

As mentioned above, Back End Services operate autonomously without user interaction. As such, they do not have a typical user/password authentication flow. Instead, backend services use client credentials which are comprised of a public/private JWKS key pair.

To create a backend services application in the Developer Platform, proceed as you would for any SMART application by adding descriptions and other metainformation, then select Backend Services as the application type:



Application Launch Type

Next, input a URL where your ES384 JWKS Public Key resides:

> **JWKS Signature Algorithm**
>
> Use the ES384 (ECDSA using P-384 and SHA-384) signature algorithm when generating your keys.

JWK URL

Finally, select the desired scopes and click **Save**. Then, click **Submit For Review**. The submitted application information is reviewed by our Greenway team against relevant standards before getting published.

Once published, the system generates a client ID which you then use to authenticate your backend service.

Backend Client ID

For further information, or for assistance in setting up Back-end Services to meet your needs, please contact us

Updated about 1 month ago

← Bulk Export

How to host a Javascript SMART-on-FHIR app on GitHub →

Did this page help you? Yes No

© 2023 All rights reserved. Greenway Health, LLC - Terms of Service - Privacy Policy

Regulatory Compliance

Product Requirements

GREENWAY FHIR R4

AllergyIntolerance

Get AllergyIntolerances GET

Get AllergyIntolerance by ID GET

Binary

CarePlan

CareTeam

Condition

Device

DiagnosticReport

DocumentReference

Encounter

Goal

Immunization

Get AllergyIntolerances

GET https://fhir-api.fhirprod.aws.greenwayhealth.com/fhir/R4/TENANT_ID/AllergyIntolerance

The AllergyIntolerance resource is a record of a clinical assessment of an allergy or intolerance; a propensity, or a potential risk to an individual, to have an adverse reaction on future exposure to the specified substance, or class of substance.

See also:
US Core AllergyIntolerance profile

QUERY PARAMS

_has string
Return resources linked to by the given target

_id string
The ID of the resource

_lastUpdated date
Only return resources which were last updated as specified by the given range

_profile string
Search for resources which have the given profile

clinical-status string
active | inactive | resolved

code string
Code that identifies the allergy or intolerance

identifier string
External ids for this item

manifestation string
Clinical symptoms/signs associated with the Event

patient string
The patient that has the allergy or intolerance.

verification-status string
unconfirmed | confirmed | refuted | entered-in-error

RESPONSES

200
OK - The request was received and the resource has been transmitted successfully. (See Response area for examples)

400
Bad Request - An unexpected error occurred. A parameter value is missing, invalid, or improperly formatted.

401
Unauthorized - User was denied access to the FHIR resource.

403
Forbidden - Access was denied for this FHIR resource with the given credentials.

404
Not Found - The FHIR resource was not found.

500
Internal Server Error - The server encountered an unexpected condition that prevented it from fulfilling the request.

Updated about 1 year ago

← Product Requirements

Get AllergyIntolerance by ID →

Did this page help you? Yes No

LANGUAGE

Shell Node Ruby PHP Python

CURL REQUEST

```
curl --request GET \
     --url https://fhir-api.fhirprod.aws.greenwayhealth.com/
     --header 'accept: application/json'
```

RESPONSE EXAMPLES

Choose an example:

application/json

200 - AllergyIntolerance

© 2023 All rights reserved. Greenway Health, LLC - Terms of Service - Privacy Policy

Regulatory Compliance

Product Requirements

GREENWAY FHIR R4

AllergyIntolerance

Get AllergyIntolerances GET

Get AllergyIntolerance by ID GET

Binary

CarePlan

CareTeam

Condition

Device

DiagnosticReport

DocumentReference

Encounter

Goal

Immunization

Get AllergyIntolerance by ID

GET https://fhir-api.fhirprod.aws.greenwayhealth.com/fhir/R4/TENANT_ID/AllergyIntolerance/{id}

Retrieves a single instance of the resource using its FHIR ID

PATH PARAMS

id string required

RESPONSES

200
OK - The request was received and the resource has been transmitted successfully. (See Response area for examples)

400
Bad Request - An unexpected error occurred. A parameter value is missing, invalid, or improperly formatted.

401
Unauthorized - User was denied access to the FHIR resource.

403
Forbidden - Access was denied for this FHIR resource with the given credentials.

404
Not Found - The FHIR resource was not found.

500
Internal Server Error - The server encountered an unexpected condition that prevented it from fulfilling the request.

Updated about 1 year ago

← AllergyIntolerance Binary →

Did this page help you? Yes No

LANGUAGE

Shell Node Ruby PHP Python

CURL REQUEST

```
curl --request GET \
     --url https://fhir-api.fhirprod.aws.greenwayhealth.com/
     --header 'accept: application/json'
```

RESPONSE EXAMPLES

Choose an example:

application/json

200 - AllergyIntolerance

© 2023 All rights reserved. Greenway Health, LLC - Terms of Service - Privacy Policy

Regulatory Compliance
Product Requirements

GREENWAY FHIR R4
- AllergyIntolerance
- Binary
  - Get Binary by ID GET
- CarePlan
- CareTeam
- Condition
- Device
- DiagnosticReport
- DocumentReference
- Encounter
- Goal
- Immunization
- Location
- MedicationRequest

Get Binary by ID

GET https://fhir-api.fhirprod.aws.greenwayhealth.com/fhir/R4/TENANT_ID/Binary/{id}

Represents a single item's raw data as digital content. A binary resource can contain any type of content, such as text, image, pdf, or zip archive. Typically, Binary resources are used for managing content, such as:

- CDA Documents
- PDF Documents
- Images

Note: Retrieval of a Binary resource follows the FHIR asynchronous request pattern.

See also:
FHIR R4 Asynchronous Request Pattern
FHIR R4 Binary Resource

PATH PARAMS

id string required

RESPONSES

200
OK - The request was received and the resource has been transmitted successfully. (See Response area for examples)

202
Accepted - The request has been accepted for processing, but the processing has not been completed.

400
Bad Request - An unexpected error occurred. A parameter value is missing, invalid, or improperly formatted.

401
Unauthorized - User was denied access to the FHIR resource.

403
Forbidden - Access was denied for this FHIR resource with the given credentials.

404
Not Found - The FHIR resource was not found.

500
Internal Server Error - The server encountered an unexpected condition that prevented it from fulfilling the request.

Updated about 1 year ago

← Get AllergyIntolerance by ID CarePlan →

Did this page help you? Yes No

LANGUAGE

Shell Node Ruby PHP Python

CURL REQUEST

```
curl --request GET \
     --url https://fhir-api.fhirprod.aws.greenwayhealth.com/
     --header 'accept: application/json'
```

RESPONSE EXAMPLES

Choose an example:

application/json
200 - Binary

/
202

© 2023 All rights reserved. Greenway Health, LLC - Terms of Service - Privacy Policy

Regulatory Compliance
Product Requirements

GREENWAY FHIR R4
- AllergyIntolerance
- Binary
- CarePlan
  - Get CarePlans GET
  - Get CarePlan by ID GET
- CareTeam
- Condition
- Device
- DiagnosticReport
- DocumentReference
- Encounter
- Goal
- Immunization
- Location

Get CarePlans

GET https://fhir-api.fhirprod.aws.greenwayhealth.com/fhir/R4/TENANT_ID/CarePlan

Care Plans are used in many areas of healthcare with a variety of scopes. They can be as simple as a general practitioner keeping track of when their patient is next due for a tetanus immunization through to a detailed plan for an oncology patient covering diet, chemotherapy, radiation, lab work and counseling with detailed timing relationships, pre-conditions and goals.

See also:
US Core CarePlan Profile

QUERY PARAMS

_has string
Return resources linked to by the given target

_id string
The ID of the resource

_lastUpdated date
Only return resources which were last updated as specified by the given range

_profile string
Search for resources which have the given profile

category string
Type of plan

encounter string
Encounter during which the care plan was created

goal string
Desired outcome of plan

identifier string
External Ids for this plan

intent string
proposal | plan | order | option

patient string
Who the care plan is for

status string
draft | active | on-hold | revoked | completed | entered-in-error | unknown

subject string
Who the care plan is for

RESPONSES

200
OK - The request was received and the resource has been transmitted successfully. (See Response area for examples)

400
Bad Request - An unexpected error occurred. A parameter value is missing, invalid, or improperly formatted.

401
Unauthorized - User was denied access to the FHIR resource.

403
Forbidden - Access was denied for this FHIR resource with the given credentials.

404
Not Found - The FHIR resource was not found.

500
Internal Server Error - The server encountered an unexpected condition that prevented it from fulfilling the request.

Updated about 1 year ago

← Get Binary by ID Get CarePlan by ID →

Did this page help you? Yes No

LANGUAGE

Shell Node Ruby PHP Python

CURL REQUEST

```
curl --request GET \
     --url https://fhir-api.fhirprod.aws.greenwayhealth.com/
     --header 'accept: application/json'
```

RESPONSE EXAMPLES

Choose an example:

application/json

200 - CarePlan

© 2023 All rights reserved. Greenway Health, LLC - Terms of Service - Privacy Policy

Regulatory Compliance
Product Requirements

GREENWAY FHIR R4
- AllergyIntolerance
- Binary
- CarePlan
  - Get CarePlans GET
  - Get CarePlan by ID GET
- CareTeam
- Condition
- Device
- DiagnosticReport
- DocumentReference
- Encounter
- Goal
- Immunization
- Location

Get CarePlan by ID

GET https://fhir-api.fhirprod.aws.greenwayhealth.com/fhir/R4/TENANT_ID/CarePlan/{id}

Retrieves a single instance of the resource using its FHIR ID

PATH PARAMS

id string required

RESPONSES

200
OK - The request was received and the resource has been transmitted successfully. (See Response area for examples)

400
Bad Request - An unexpected error occurred. A parameter value is missing, invalid, or improperly formatted.

401
Unauthorized - User was denied access to the FHIR resource.

403
Forbidden - Access was denied for this FHIR resource with the given credentials.

404
Not Found - The FHIR resource was not found.

500
Internal Server Error - The server encountered an unexpected condition that prevented it from fulfilling the request.

Updated about 1 year ago

← CarePlan CareTeam →

Did this page help you? Yes No

LANGUAGE

Shell Node Ruby PHP Python

CURL REQUEST

```
curl --request GET \
     --url https://fhir-api.fhirprod.aws.greenwayhealth.com/
     --header 'accept: application/json'
```

RESPONSE EXAMPLES

Choose an example:

application/json

200 - CarePlan

© 2023 All rights reserved. Greenway Health, LLC - Terms of Service - Privacy Policy

Regulatory Compliance
Product Requirements

GREENWAY FHIR R4
- AllergyIntolerance
- Binary
- CarePlan
- CareTeam
  - Get CareTeams GET
  - Get CareTeam by ID GET
- Condition
- Device
- DiagnosticReport
- DocumentReference
- Encounter
- Goal
- Immunization
- Location

Get CareTeams

GET https://fhir-api.fhirprod.aws.greenwayhealth.com/fhir/R4/TENANT_ID/CareTeam

The CareTeam includes all the people, teams, and organizations who plan to participate in the coordination and delivery of care for a single patient or a group (such as a married couple in therapy or a support group). This is not limited to practitioners, but may include other caregivers such as family members, guardians, the patient themself, or others. The Care Team, depending on where used, may include care team members specific to a particular care plan, an episode, an encounter, or may reflect all known team members across these perspectives.

See also:
US Core CareTeam Profile

QUERY PARAMS

_has string
Return resources linked to by the given target

_id string
The ID of the resource

_lastUpdated date
Only return resources which were last updated as specified by the given range

_profile string
Search for resources which have the given profile

category string
Type of team

identifier string
External Ids for this team

participant string
Who is involved

patient string
Who care team is for

status string
proposed | active | suspended | inactive | entered-in-error

subject string
Who care team is for

RESPONSES

200
OK - The request was received and the resource has been transmitted successfully. (See Response area for examples)

400
Bad Request - An unexpected error occurred. A parameter value is missing, invalid, or improperly formatted.

401
Unauthorized - User was denied access to the FHIR resource.

403
Forbidden - Access was denied for this FHIR resource with the given credentials.

404
Not Found - The FHIR resource was not found.

500
Internal Server Error - The server encountered an unexpected condition that prevented it from fulfilling the request.

Updated about 1 year ago

← Get CarePlan by ID Get CareTeam by ID →

Did this page help you? Yes No

LANGUAGE

Shell Node Ruby PHP Python

CURL REQUEST

```
curl --request GET \
     --url https://fhir-api.fhirprod.aws.greenwayhealth.com/
     --header 'accept: application/json'
```

RESPONSE EXAMPLES

Choose an example:

application/json

200 - CareTeam

© 2023 All rights reserved. Greenway Health, LLC - Terms of Service - Privacy Policy